Synopsis Report

"KioMeet — A Secure Native Interview & Video Conferencing Platform"

by

Aman Verma

23bcs012

Ankit Kashyap

23bcs017

Amrit Kashyap

23bcs014

Under the guidance of Dr. Manoj Gupta

Submitted in partial fulfilment of the requirements for the award of the degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING



SHRI MATA VAISHNO DEVI UNIVERSITY, KATRA

(School of Computer Science & Engineering)

JAMMU & KASHMIR – 182 320

Session 2026–27 (Odd)

INTRODUCTION

Remote hiring and distributed teamwork have made video conferencing a daily necessity. Yet most general-purpose meeting tools are browser- or Electron-based, which limits access to native operating-system capabilities, adds overhead, and — critically for technical hiring — offers no reliable way to stop private interviewer material from leaking into the shared video stream.

This project proposes **KioMeet**, a native cross-platform video conferencing application for **Windows and macOS**, built specifically for teams and technical interviews. KioMeet runs as a native desktop application on each platform (Objective-C and Swift on macOS; C++/.NET on Windows) with no browser or Electron layer, pairing a GPU-accelerated media pipeline with a dedicated Node.js backend for room management, authentication, scheduling, recording, and team collaboration.

KioMeet's distinguishing feature is a dedicated **secure interview mode**. In this mode the interviewer's own screen is rendered inside a hardened window whose contents are never captured into the outgoing stream; only the content of designated tool surfaces (code editor, whiteboard) is shared with the candidate. This guarantees that private questions, notes, browser tabs, or other application windows never bleed into the call, preserving interview integrity while keeping the experience fast and natural.

System Overview

KioMeet is organised in three cooperating layers. The *native desktop client* (Windows and macOS) owns media capture, GPU rendering (Metal on macOS, Direct3D on Windows), the secure window, and the user interface. The *real-time media layer* uses a LiveKit-compatible WebRTC server [1, 2] to publish and subscribe audio, video, and screen-share tracks. The *backend token server* (Node.js + Express) issues short-lived access tokens, manages rooms, enforces user tiers, schedules meetings, syncs calendars, and tracks recordings, backed by PostgreSQL for persistent data and Redis for ephemeral room state.

LITERATURE SURVEY

1. WebRTC and the LiveKit SFU model

WebRTC is the open standard for browser- and application-based real-time communication [1, 5]. At scale, mesh topologies become impractical, so modern platforms adopt a Selective Forwarding Unit (SFU) such as LiveKit [2], which routes media without transcoding and supports adaptive simulcast. This body of work shows that an SFU is the appropriate architecture for rooms of up to tens of participants while keeping client CPU and bandwidth bounded.

2. Native screen-capture APIs (Windows & macOS)

Both desktop platforms expose modern, permission-gated screen-capture APIs: Apple's ScreenCaptureKit on macOS [3] and the Windows Graphics Capture API on Windows [4], each supporting display and individual-window capture with live thumbnails and system-audio capture. Compared with legacy capture paths they offer lower latency, finer-grained window selection, and explicit user consent. This establishes that native frameworks on each OS can deliver screen sharing that is both higher quality and more privacy-respecting than cross-platform wrappers.

3. Secure window composition and content isolation

Research and platform documentation on window-server composition [3, 4] show that an application can place a window above the normal layer order and selectively exclude surfaces from capture. Applied to interviewing, this means the interviewer's private material can be isolated from the shared stream at the compositor level, so only explicitly designated tool surfaces are ever transmitted — a stronger guarantee than hiding windows in software alone.

4. Token-based authentication and tiered access

Established security practice favours a short-lived JWT access token [6] paired with a rotating, hashed refresh token, with the user's service tier re-read from the database on every refresh rather than trusted from the client. This literature underpins KioMeet's auth model and its fail-closed, event-driven billing and tier enforcement.

SUMMARY OF LITERATURE SURVEY

1. A native client built on an SFU (LiveKit) and each platform's modern frameworks (ScreenCaptureKit/Metal on macOS, Windows Graphics Capture/Direct3D on Windows) delivers lower-latency, higher-quality conferencing than browser or Electron alternatives.
2. Content isolation for interviews is best enforced at the window-server / capture-path level, not by hiding windows in application code.
3. A short-lived access token plus rotating refresh token, with tier always re-read server-side, is the accepted pattern for secure, tier-gated access.

PROBLEM STATEMENT

To design and build a native cross-platform (Windows and macOS) video conferencing platform that supports both standard team calls and technical interviews, in which the interviewer's private material is provably isolated from the shared video stream, while delivering reliable multi-participant audio/video, screen sharing, recording, scheduling, and moderation through a secure, tier-gated backend — without the overhead and capability limits of browser- or Electron-based solutions.

OBJECTIVES

The specific objectives of this project are as follows:

1. To build native desktop clients for **Windows and macOS** (Objective-C/Swift on macOS; C++/.NET on Windows) that capture camera, microphone, and screen content and render remote participants on the GPU (Metal on macOS, Direct3D on Windows), scaling the layout adaptively up to 50 participants.
2. To implement a *secure interview mode* in which the interviewer's screen is composed into a hardened window and only designated tool surfaces (code editor, whiteboard) are streamed to the candidate, preventing any content leakage.
3. To integrate a LiveKit-compatible WebRTC server for publishing and subscribing audio, video, and screen-share tracks, with decoupled audio/video paths so a microphone toggle never interrupts the camera.
4. To develop a Node.js backend that issues short-lived JWT access tokens with rotating refresh tokens, manages rooms (Redis, 24-hour TTL), and stores persistent data in PostgreSQL.
5. To provide meeting features — public and direct chat, raise-hand and speaker queue, live polls, a Q&A board, role-based moderation, a lobby/waiting room, and local, cloud, and multi-track recording.
6. To support scheduling with email invitations (`.ics`) and Apple, Google, and Outlook calendar sync, and tier-gated access (free, pro, hiring) that is enforced server-side and fail-closed.

METHODOLOGY

1. Requirement Analysis

- Identify needs for team calls and technical interviews (isolation, recording, moderation, scheduling).
- Study existing platforms (Zoom, Google Meet, native WebRTC apps) and the LiveKit SFU model.

2. System Design

- **Native clients (Windows & macOS)** — media capture, GPU rendering (Metal/Direct3D), secure window, and UI panels.
- **Media layer** — LiveKit publisher/subscriber bridging local and remote tracks.
- **Backend** — Express token server with PostgreSQL (persistent) and Redis (ephemeral room state).

3. Core Client Development (Windows & macOS)

- Camera and microphone capture with two-phase toggles that eliminate frozen/black frames on mute.
- Screen and window sharing via each platform's native capture API (ScreenCaptureKit on macOS, Windows Graphics Capture on Windows) with a live-thumbnail picker and optional system-audio capture.
- Secure interview mode: interviewer content composed into a hardened window; only designated tool surfaces are published to the candidate.
- Adaptive grid, stage + filmstrip, and spotlight layouts; active-speaker highlighting and auto-spotlight.

4. Backend & Real-Time Services

- JWT access tokens (short TTL) with rotating, hashed refresh tokens and family-based theft detection.
- Room creation and 6-character join codes with 24-hour expiry in Redis.

- Scheduling with `.ics` email invites and Google/Outlook OAuth calendar sync; team management with role-based permissions.
- Local, cloud (S3 egress), and multi-track recording with a composed renderer and disk-space safeguards.

5. Testing, Deployment & Verification

- Unit and integration tests on each platform (XCTest on macOS and the equivalent on Windows) covering tokens, media bridges, room lifecycle, isolation invariants, and recording.
- Health-checked backend deployment (PostgreSQL + Redis) and end-to-end verification between live Windows and macOS clients.

EXPECTED RESULTS

1. **Reliable native conferencing** — smooth multi-participant audio/video with GPU rendering and adaptive layouts, outperforming browser/Electron alternatives in latency and resource use.
2. **Guaranteed interview isolation** — in secure interview mode, only designated tool surfaces reach the candidate; the interviewer's other windows and material never appear in the stream.
3. **Full collaboration suite** — chat, polls, Q&A, raise-hand queue, moderation, lobby, scheduling with calendar sync, teams, and local/cloud/multi-track recording working end-to-end.
4. **Secure, tier-gated backend** — short-lived tokens, rotating refresh tokens, and fail-closed tier enforcement protecting all features and data.

TIME SCHEDULE

Phase	**Duration**	**Timeline**
Phase 1: Literature Review & Requirements	2 weeks	1 Sept – 14 Sept 2026
Phase 2: System Design	2 weeks	15 Sept – 28 Sept 2026
Phase 3: Client & Backend Implementation	4 weeks	29 Sept – 26 Oct 2026
Phase 4: Integration (media, secure mode)	2 weeks	27 Oct – 9 Nov 2026
Phase 5: Testing & Debugging	2 weeks	10 Nov – 23 Nov 2026
Phase 6: Documentation & Finalization	1 week	24 Nov – 30 Nov 2026
Total Project Duration	**13 weeks**	**1 Sept – 30 Nov 2026**

REFERENCES

[1] WebRTC — Official Project Documentation. `https://webrtc.org`

[2] LiveKit — Open-Source WebRTC SFU Documentation. `https://docs.livekit.io`

[3] Apple Developer Documentation — ScreenCaptureKit & Metal (macOS). `https://developer.apple.com/documentation/screencapturekit`

[4] Microsoft Documentation — Windows Graphics Capture & Direct3D (Windows). `https://learn.microsoft.com/windows/uwp/audio-video-camera/screen-capture`

[5] IETF RFC 8825 — Overview: Real-Time Protocols for Browser-Based Applications (WebRTC).

[6] IETF RFC 7519 — JSON Web Token (JWT).

Student Sign **Guide Sign**

1. Aman Verma
2. Ankit Kashyap
3. Amrit Kashyap